SQL — Structured Query Language

Database Management · GATE CS · Placements · Backend Development

CHEAT SHEET | HIGH YIELD | EXAM READY

CORE CONCEPTS

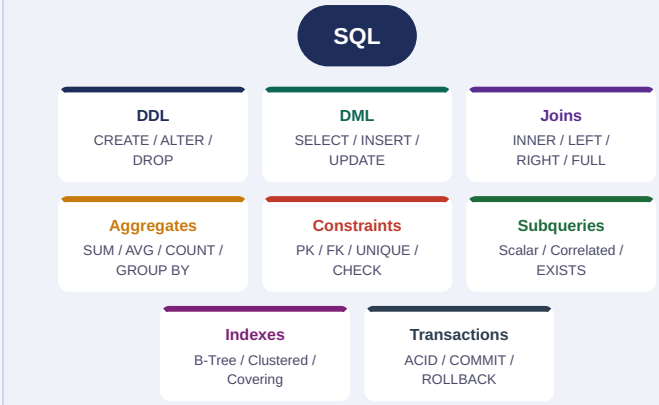
Definitions · Big Picture · Key Terminology

Foundational Definitions

- **SQL:** Declarative language for managing relational databases — you say *what*, not *how*
- **RDBMS:** Stores data in tables (rows + columns) with enforced relationships
- **Schema:** Blueprint of the database — tables, columns, types, constraints
- **Primary Key:** Uniquely identifies each row; NOT NULL + UNIQUE enforced
- **Foreign Key:** Links a column to a PK in another table; enforces referential integrity
- **NULL:** Absence of value — not zero, not empty string; propagates in expressions

SQL Command Categories

- **DDL** (Data Definition): `CREATE, ALTER, DROP, TRUNCATE`
- **DML** (Data Manipulation): `SELECT, INSERT, UPDATE, DELETE`
- **DCL** (Data Control): `GRANT, REVOKE`
- **TCL** (Transaction Control): `COMMIT, ROLLBACK, SAVEPOINT`
- **DQL** (Data Query): `SELECT` — often merged into DML



{ } ESSENTIAL SQL SYNTAX

SELECT · Joins · Aggregates · Subqueries · DDL

SELECT — FULL CLAUSE ORDER

```
SELECT col1, col2
FROM   table
WHERE  condition
GROUP BY col
HAVING agg_condition
ORDER BY col DESC
LIMIT  n;
```

Execution order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER → LIMIT

JOINS — SYNTAX PATTERN

```
SELECT *
FROM A
  INNER JOIN B
    ON A.id = B.a_id;

-- LEFT: all A + matched B
-- RIGHT: all B + matched A
-- FULL: all rows both sides
-- CROSS: every combination
```

INNER: only matching rows in BOTH tables

AGGREGATE FUNCTIONS

```
COUNT(*)        -- all rows
COUNT(col)      -- non-NULL only
SUM(col)
AVG(col)
MAX(col)
MIN(col)
GROUP_CONCAT(col)
```

HAVING filters groups (post-aggregation); WHERE filters rows (pre-aggregation)

SUBQUERY PATTERNS

```
-- Scalar subquery
SELECT (SELECT MAX(sal) FROM
emp);

-- IN subquery
WHERE id IN (SELECT id FROM
dept);

-- EXISTS
WHERE EXISTS (SELECT 1 FROM B
  WHERE B.id=A.id);
```

Correlated subquery: references outer query — runs once per outer row

WINDOW FUNCTIONS

```
ROW_NUMBER() OVER (
  PARTITION BY dept
  ORDER BY salary DESC
)

RANK()        -- ties get same
rank
DENSE_RANK()  -- no gap after
ties
LAG(col,1)    -- previous row
LEAD(col,1)   -- next row
```

OVER(): defines window; PARTITION BY = group; ORDER BY = sort within group

DDL & DML ESSENTIALS

```
CREATE TABLE t (
  id   INT PRIMARY KEY,
  name VARCHAR(50) NOT NULL
);
ALTER TABLE t ADD col INT;
INSERT INTO t VALUES(1,'A');
UPDATE t SET name='B' WHERE id=1;
DELETE FROM t WHERE id=1;
```

TRUNCATE removes all rows faster than DELETE (no WHERE, no rollback in some DBs)

⚙ QUERY EXECUTION FLOW

How SQL actually processes a SELECT statement

SELECT Statement — Logical Execution Order

Write Query: SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY

↓

```
1. FROM / JOIN — load tables, apply join conditions
```

↓

```
2. WHERE — filter individual rows (cannot use aliases or aggregates)
```

↓

```
3. GROUP BY — group remaining rows by specified columns
```

↓

```
4. HAVING — filter groups (CAN use aggregate functions here)
```

↓

```
5. SELECT — evaluate expressions, apply aliases, DISTINCT
```

↓

```
6. ORDER BY — sort result set (can use aliases from SELECT)
```

↓

7. LIMIT / OFFSET — return final n rows

⊞ COMPARISON TABLES

JOINs · WHERE vs HAVING · DELETE vs TRUNCATE · RANK functions

JOIN Types — Side by Side

JOIN TYPE	RETURNS	NULL ROWS?
INNER JOIN	Matching rows in BOTH tables	No
LEFT JOIN	All left + matched right	Right cols = NULL
RIGHT JOIN	All right + matched left	Left cols = NULL
FULL OUTER	All rows from both	Both sides possible
CROSS JOIN	Cartesian product (m×n rows)	No
SELF JOIN	**Table joined with itself**	**Depends on type**

RANK vs DENSE_RANK vs ROW_NUMBER

SCORE	ROW_NUMBER	RANK	DENSE_RANK
100	1	1	1
100	2	1	1
90	3	3	2
80	4	4	3

RANK skips numbers after ties; DENSE_RANK never skips

DELETE vs TRUNCATE vs DROP

FEATURE	DELETE	TRUNCATE	DROP
Type	DML	DDL	DDL
WHERE clause	Yes	No	No
Rollback	Yes	Sometimes	No
Table exists after	Yes	Yes (empty)	No
Speed	Slow	Fast	Instant

WHERE vs HAVING

PROPERTY	WHERE	HAVING
Filters	Individual rows	Groups of rows
Execution step	Before GROUP BY	After GROUP BY
Use aggregates?	No	Yes
Use aliases?	No	Sometimes (DB-specific)
Example	**WHERE age > 18**	**HAVING COUNT(*) > 5**

🧠 MNEMONICS & MEMORY TRICKS

Recall clause order, JOIN types, ACID instantly

SQL CLAUSE EXECUTION ORDER

FW-GH-SOL

- **F** — **FROM** — load the table(s)
- **W** — **WHERE** — filter rows
- **G** — **GROUP BY** — group rows
- **H** — **HAVING** — filter groups
- **S** — **SELECT** — pick columns
- **O** — **ORDER BY** — sort results
- **L** — **LIMIT** — cap output

ACID PROPERTIES OF TRANSACTIONS

ACID

- **A** — **Atomicity** — all or nothing
- **C** — **Consistency** — valid state always
- **I** — **Isolation** — concurrent txns don't interfere
- **D** — **Durability** — committed data survives crash

SQL COMMAND CATEGORIES

DD-DM-DC-TC

- **DD** — **DDL** — CREATE, ALTER, DROP
- **DM** — **DML** — SELECT, INSERT, UPDATE, DELETE
- **DC** — **DCL** — GRANT, REVOKE
- **TC** — **TCL** — COMMIT, ROLLBACK, SAVEPOINT

⚠ COMMON MISTAKES & EXAMINER TRAPS

Classic errors that cost marks

⚠ QUERY LOGIC TRAPS

- **NULL comparisons:** `col = NULL` is ALWAYS false — use `IS NULL` or `IS NOT NULL`
- **COUNT(*) vs COUNT(col):** COUNT(*) counts all rows; COUNT(col) skips NULLs
- **WHERE with aggregates:** Using `WHERE COUNT(*) > 5` is INVALID — use HAVING
- **SELECT clause order:** Writing clauses in wrong order causes syntax error — execution order ≠ write order
- **DISTINCT with COUNT:** `COUNT(DISTINCT col)` counts unique non-NULL values only

⚠ JOIN & KEY TRAPS

- **Missing JOIN condition:** Forgetting ON clause creates a CROSS JOIN (m×n rows)
- **INNER vs LEFT mix-up:** If right table has no match, INNER removes the row; LEFT keeps it with NULLs
- **GROUP BY missing columns:** Every non-aggregate in SELECT must appear in GROUP BY (MySQL is lenient, others are not)
- **Subquery vs JOIN performance:** Correlated subquery re-executes per row — JOIN is almost always faster
- **TRUNCATE is DDL:** Cannot be rolled back in many databases — not reversible like DELETE

✏ WORKED MINI-EXAMPLES

Pattern recognition — crack the method, not just the answer

Example 1 — 2nd Highest Salary (Classic Interview Q)

Find the 2nd highest salary from the Employee table.

1. Sort salaries descending and pick 2nd: use LIMIT with OFFSET
2. Or use subquery: exclude the MAX, find MAX of remainder
3. Best approach: use DENSE_RANK for ties-safe result

```
SELECT salary FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS r FROM emp) t WHERE r = 2;
```

Example 2 — Departments with >5 Employees

List department names where more than 5 employees work.

1. JOIN Employee and Department tables on dept_id
2. GROUP BY department name
3. Filter groups with HAVING COUNT(*) > 5

```
SELECT d.name FROM dept d JOIN emp e ON d.id=e.dept_id GROUP BY d.name HAVING COUNT(*) > 5;
```

Example 3 — Employees with No Manager (Self Join)

Find all employees who have no manager in the same table (manager_id IS NULL or not in emp table).

1. Use LEFT JOIN of emp table with itself on manager_id = id
2. Filter where the manager side returns NULL
3. Alternatively: WHERE manager_id IS NULL

```
SELECT e.name FROM emp e LEFT JOIN emp m ON e.manager_id = m.id WHERE m.id IS NULL;
```

Example 4 — Cumulative Sum (Window Function)

Calculate running total of sales ordered by date per region.

1. Use SUM() as a window function with OVER()
2. PARTITION BY region, ORDER BY sale_date inside OVER()
3. Add ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW for explicit frame

```
SELECT region, date, SUM(sales) OVER (PARTITION BY region ORDER BY date) AS cum_sales FROM sales;
```

🎯 EXAM STRATEGY & HIGH-YIELD TIPS

Score maximizers · Placement patterns · GATE focus

High-Yield Exam Patterns

- **Nth highest salary:** Always asked in placements — use DENSE_RANK() OVER (ORDER BY salary DESC) WHERE rank = N
- **Self-join questions:** Employee-Manager hierarchy — recognize the pattern: same table, different aliases
- **GROUP BY + HAVING combo:** Most common aggregate question type — find groups satisfying a count/sum condition
- **NULL handling:** IS NULL, COALESCE(col, default), NULLIF(a,b) — frequently tested edge case
- **Execution order MCQ:** GATE loves asking "which clause executes first" — memorize FW-GH-SOL

Speed Tricks & Score Boosters

- **EXISTS vs IN:** EXISTS stops at first match (faster on large sets); IN evaluates all values
- **COALESCE shortcut:** `COALESCE(col, 0)` replaces NULL with 0 — avoids NULL propagation in math
- **CASE WHEN:** Used for conditional aggregation — `SUM(CASE WHEN status='A' THEN 1 ELSE 0 END)`
- **Index hint:** B-Tree indexes help on equality and range; never index low-cardinality columns (like gender)
- **Elimination trick:** If two JOIN options are given, ask "does the question need unmatched rows?" — if yes, INNER JOIN is wrong

⚡ QUICK REFERENCE

String · Date · Numeric functions at a glance

String Functions

- `UPPER(s)` — convert to uppercase
- `LOWER(s)` — convert to lowercase
- `LENGTH(s)` — character count
- `SUBSTRING(s,pos,len)` — extract part
- `TRIM(s)` — remove leading/trailing spaces
- `CONCAT(a,b)` — join strings
- `REPLACE(s,old,new)` — substitution
- `LIKE '%pat%'` — pattern match (% = any chars, _ = one char)

Date & Time Functions

- `NOW()` — current datetime
- `CURDATE()` — current date only
- `DATE_ADD(d, INTERVAL n DAY)`
- `DATEDIFF(d1, d2)` — days between
- `YEAR(d) / MONTH(d) / DAY(d)`
- `DATE_FORMAT(d, '%Y-%m')`
- `EXTRACT(MONTH FROM d)`

Numeric & Constraints

- `ROUND(n, decimals)` — round number
- `FLOOR(n) / CEIL(n)`
- `ABS(n)` — absolute value
- `MOD(a, b)` — remainder
- **UNIQUE:** no duplicate values in column
- **CHECK:** validates column values against condition
- **DEFAULT:** auto-fills if no value given
- **NOT NULL:** column must always have a value